

INSTITUTO FEDERAL
SÃO PAULO



Programação Assíncrona com Dart e Operações de Rede com Flutter

Programação para Dispositivos Móveis (PDMI6)

Prof. Luiz Gustavo Diniz de Oliveira Vêras

E-mail: gustavo_veras@ifsp.edu.br



Conteúdo

- Dart: Future e Streams
- Fazendo requisições HTTP com Flutter

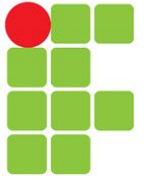


Programação assíncrona em Dart

Por que o código assíncrono é importante

As operações assíncronas permitem que seu programa conclua o trabalho enquanto aguarda a conclusão de outra operação. Aqui estão algumas operações assíncronas comuns:

- Buscando dados em uma rede.
- Gravando em um banco de dados.
- Lendo dados de um arquivo.



Programação assíncrona em Dart

Esses cálculos assíncronos geralmente fornecem seu resultado como um **Future** ou, se o resultado tiver várias partes, como um **Stream**.

Para interagir com esses resultados assíncronos, você pode usar as palavras-chave *async* e *await* .

A maioria das funções assíncronas são apenas funções assíncronas do Dart que dependem, possivelmente no fundo, de uma computação inerentemente assíncrona.



Future

O que é um Future?

Um *future* (minúsculo “f”) é uma instância da classe *Future* (maiúsculo “F”). Um *future* representa o resultado de uma operação assíncrona e pode ter dois estados: **não concluído/Incompleto** ou **concluído**.

Incompleto é um termo do Dart que se refere ao estado de um futuro antes de produzir um valor

Se a operação assíncrona for bem-sucedida, o futuro será concluído com um valor. Caso contrário, ele será concluído com um erro.



Future

Uma operação assíncrona do tipo *Future<T>* é concluída com um valor do tipo *T*, que é especificada pelo programador. Por exemplo, um *future* com o tipo *Future<String>* produz um valor de string. Se um *future* não produz um valor utilizável, então o tipo do futuro é *Future<void>*.

```
Future<void> fetchUserOrder() {  
    // Imagine que esta função está buscando dados  
    // de usuário de outro serviço ou banco de dados  
    Future.delayed(  
        const Duration(seconds: 2),  
        () => print('Large Latte'),  
    );  
}  
  
void main() {  
    fetchUserOrder();  
    print('Buscando pedido de usuário...');  
}
```

Future



```
Future<void> fetchUserOrder() {  
    // Imagine que esta função está buscando dados  
    // de usuário de outro serviço ou banco de dados  
    Future.delayed(  
        const Duration(seconds: 2),  
        () => print('Large Latte'),  
    );  
}  
  
void main() {  
    fetchUserOrder();  
    print('Buscando pedido de usuário...');  
}
```

No exemplo, embora **fetchUserOrder()** seja executado antes da chamada `print()`, o console mostra a saída da String (“Buscando a ordem do usuário...”) antes da saída de **fetchUserOrder()** (“Large Latte”). Isso ocorre porque `fetchUserOrder()` demora mais para imprimir “Large Latte”.



Future

```
Future<void> fetchUserOrder() {
    // Imagine que esta função retorna um Future
    // de usuário de outro sistema
    Future.delayed(
        const Duration(seconds: 2),
        () => print('Large at')
    );
}

void main() {
    fetchUserOrder();
    print('Buscando pedido')
}
```

Você aprendeu sobre *futures* e como eles são concluídos, mas como você usa os resultados de funções assíncronas?

... executado
... int(), o console
... ing ("Buscando
...") antes da
... der() ("Large
...orque
...hora mais para
...".



Future

As palavras-chave `async` e `await` fornecem uma maneira declarativa de definir funções assíncronas e usar seus resultados. Diretrizes ao usar **`async`** e **`await`**: Para definir uma função assíncrona, adicione `async` antes do corpo da função:

```
void main() async { ... }
```

A palavra-chave `await` funciona apenas em funções assíncronas.

```
print(await createOrderMessage());
```



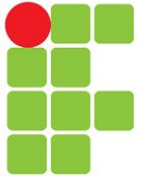
Future

Se a função tiver um tipo de retorno declarado, atualize o tipo para ser `Future<T>`, onde `T` é o tipo de valor que a função retorna: `Future<String>`, `Future<int>`, etc.

Se a função não retorna explicitamente um valor, então o tipo de retorno é *Future<void>*:

```
Future<void> main() async { ... }
```

Diferença entre função síncrona e assíncrona



```
String createOrderMessage() {  
    var order = fetchUserOrder();  
    return 'Seu pedido é: $order';  
}  
  
Future<String> fetchUserOrder() {  
    // Imagine que ésta é uma função  
    // lenta e complexa.  
    Future.delayed(  
        const Duration(seconds: 2),  
        () => 'Large Latte',  
    );  
}  
  
void main() {  
    print(createOrderMessage());  
    print('Buscando pedido de usuário...');  
}
```

```
PS C:\dart_projs> dart .\synchronous.dart  
Buscando pedido de usuário...  
Seu pedido é: Instance of 'Future<String>'
```

Diferença entre função síncrona e assíncrona



```
Future<String> createOrderMessage() async {  
  var order = await fetchUserOrder();  
  return 'Seu pedido é: $order';  
}  
  
Future<String> fetchUserOrder() {  
  // Imagine que ésta é uma função  
  // lenta e complexa.  
  Future.delayed(  
    const Duration(seconds: 2),  
    () => 'Large Latte',  
  );  
}  
  
Future<void> main () async{  
  print(await createOrderMessage());  
  print('Buscando pedido de usuário...');  
}
```

Agora estamos utilizando **await** antes de **fetchUserOrder()**. Assim a função será executada antes do **print()**. A função **main** passa a ser uma função **async** agora (ou seja, ela retornará um *Future*).

```
PS C:\dart_projs> dart .\asynchronous.dart  
Buscando pedido de usuário...  
Seu pedido é: Large Latte
```

Diferença entre função síncrona e assíncrona



```
Future<String> createOrderMessage() async {  
  var order = await fetchUserOrder();  
  return 'Seu pedido é: $order';  
}  
  
Future<String> fetchUserOrder() {  
  // Imagine que ésta é uma função  
  // lenta e complexa.  
  Future.delayed(  
    const Duration(seconds: 2),  
    () => 'Large Latte',  
  );  
}  
  
Future<void> main () async{  
  print(await createOrderMessage());  
  print('Buscando pedido de usuário...');  
}
```

Agora estamos utilizando **await** antes de **fetchUserOrder()**. Assim a função será executada antes do **print()**. A função **main** passa a ser uma função **async** agora (ou seja, ela retornará um *Future*).

```
PS C:\dart_projs> dart .\asynchronous.dart  
Buscando pedido de usuário...  
Seu pedido é: Large Latte
```



Future

Para tratar erros dentro de uma função assíncrona, utilizamos o bloco try-catch:

```
try {  
    print('Awaiting user order...');  
    var order = await fetchUserOrder();  
} catch (err) {  
    print('Caught error: $err');  
}
```



Streams

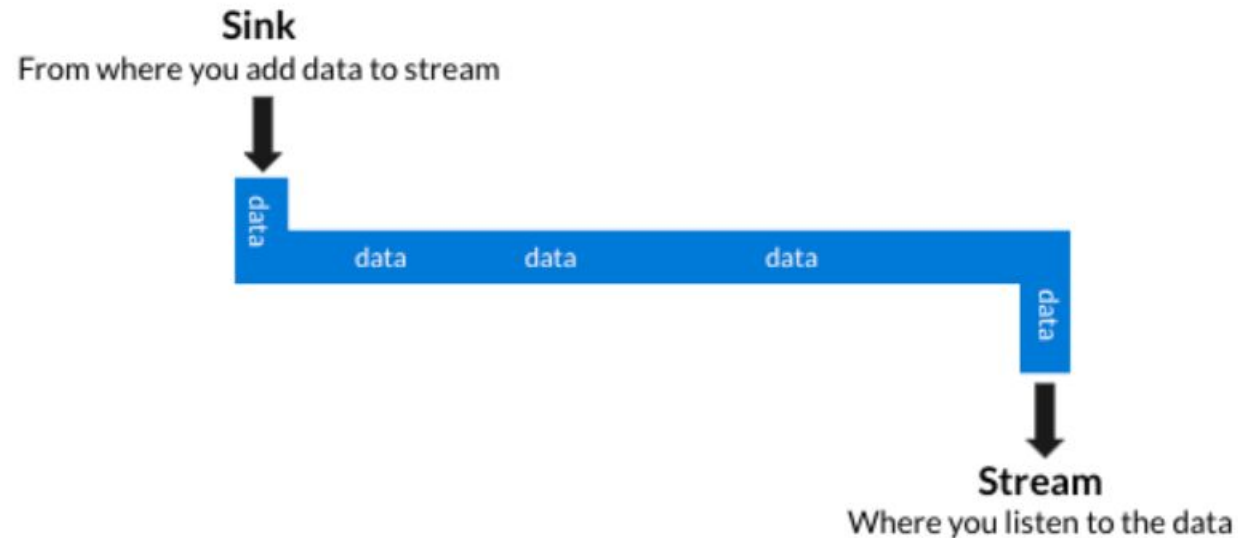
Uma **Stream** é uma sequência de eventos assíncronos. É como um iterável assíncrono — onde, em vez de receber o próximo evento quando você o solicita, o fluxo informa automaticamente que há um evento quando ele estiver pronto.

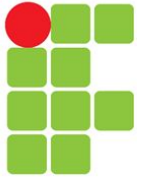
```
Stream<int> stream
```



Streams

Quando trabalhamos com ***Stream***, temos uma fonte produtora de eventos assíncronos (chamada de *Sink*) que são tratadas em um código que escuta por esses eventos (o *Stream*).





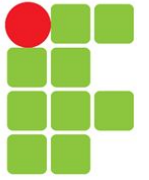
Streams

Para usarmos *stream*, precisamos definir uma função com `async`

E consumimos então a *stream* em um laço definido com `await for`

```
await for (final value in stream) {  
    sum += value;  
}
```

O *loop* assíncrono `for` (comumente chamado apenas de *await for*) itera sobre os eventos de um fluxo como o *loop for* itera sobre um *Iterable*.



Streams

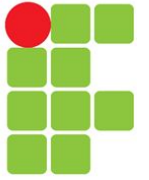
Exemplo

Gerando uma stream de inteiros com `async*`

```
Stream<int> countStream(int to) async* {  
    for (int i = 1; i <= to; i++) {  
        await Future.delayed(Duration(seconds:1));  
        yield i;  
    }  
}
```

Indica que a função é assíncrona e produzirá um iterador (ou stream) de valores assíncronos.

Dentro da função **async***, a palavra-chave **yield** é usada para pausar a execução da função e retornar um valor. Ao invés de retornar o valor diretamente, yield devolve uma Future que contém o valor **i**, permitindo que o iterador assíncrono seja criado.



Streams

Exemplo

Consumindo a stream

```
Stream<int> countStream(int to) async* {  
    for (int i = 1; i <= to; i++) {  
        await Future.delayed(Duration(seconds:1));  
        yield i;  
    }  
}  
  
void main() async {  
    await for (final value in countStream(5)) {  
        print(value); // 1, 2, 3, 4, 5  
    }  
}
```

O laço dentro de **countStream** segura a repetição até que um novo valor seja produzido pelo *stream* criado em **countStream**

Esse programa contará até 5 de forma assíncrona simulada. A cada 1 segundos o iterador dentro de **countStream** faz um laço e traz o valor seguinte.



Requisição HTTP com Flutter

Um dos usos mais comuns para o uso das classes de operações assíncronas é na realização de requisições HTTP à servidores Web.

Para demonstração, vamos estar nos conectando ao **JSONPlaceholder** (<https://jsonplaceholder.typicode.com>), uma API online usada para fins de teste. É um serviço gratuito no qual você pode enviar solicitações GET ou POST e retorna vários tipos de strings codificadas em JSON como resposta.

Para os exemplos, instale o pacote oficial do Dart para requisições web com HTTP – http (<https://pub.dev/packages/http>)

```
flutter pub add http
```



Requisição HTTP com Flutter

Atenção!

Para utilizar operações de rede da plataforma Android, realizar as configurações a seguir.

Notas da plataforma

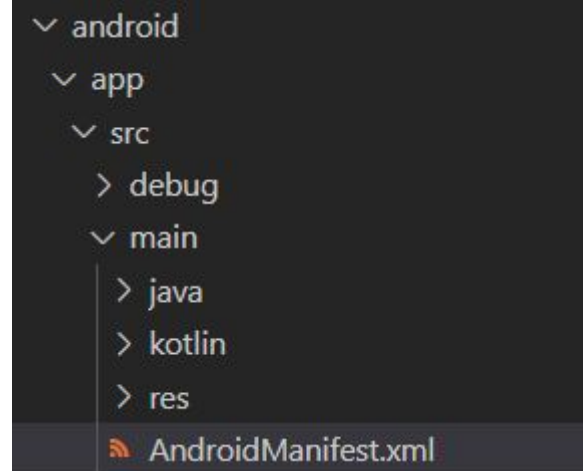
Algumas plataformas exigem etapas adicionais, conforme detalhado abaixo.

Android

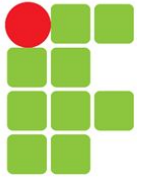
Os aplicativos Android devem [declarar seu uso da Internet](#) no manifesto do Android (`AndroidManifest.xml`):

```
<manifest xmlns:android...>
...
<uses-permission android:name="android.permission.INTERNET" />
<application ...
</manifest>
```

xml



Fonte: <https://docs.flutter.dev/data-and-backend/networking#platform-notes>



Requisição HTTP com Flutter

Para trabalhar com o módulo, é requerido a especificação de URL como segue:

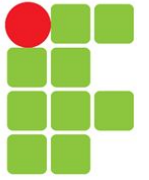
```
var url = Uri.https('jsonplaceholder.typicode.com', 'posts/1');
```

Adaptamos conforme documentação do jsonplaceholder

<https://jsonplaceholder.typicode.com/guide/>

Getting a resource

```
fetch('https://jsonplaceholder.typicode.com/posts/1')  
  .then((response) => response.json())  
  .then((json) => console.log(json));
```



Requisição HTTP com Flutter

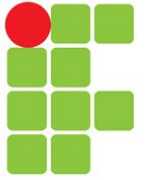
Agora podemos realizar Operações com o módulo HTTP:

Operação GET (recuperar dados)

```
var response = await http.get(url, headers: {'Content-Type': 'application/json'});  
print('Response status: ${response.statusCode}');  
print('Response body: ${response.body}');
```

Operação POST (enviar dados)

```
var response = await http.post(url, body: {'name': 'doodle', 'color': 'blue'});  
print('Response status: ${response.statusCode}');  
print('Response body: ${response.body}');
```



Requisição HTTP com Flutter

Agora podemos realizar Operações com o módulo HTTP:

Operação GET (recuperar dados)

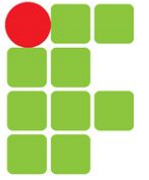
```
var response = await http.get(url, headers: {'Content-Type': 'application/json'});  
print('Response status: ${response.statusCode}');  
print('Response body: ${response.body}');
```

Devemos especificar uma representação do objeto em *json* para enviar no *body* da requisição.

Operação POST (enviar dados)

```
var response = await http.post(url, body: {'name': 'doodle', 'color': 'blue'});  
print('Response status: ${response.statusCode}');  
print('Response body: ${response.body}');
```

Os verbos HTTP PUT, PATCH e DELETE são similares.



Requisição HTTP com Flutter

Para Facilitar, você pode criar classes para representar os dados dos tipos retornados pelas APIs com métodos de conversão De JSON e Para JSON.

```
import 'dart:convert';

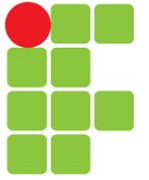
class Post {
  final int userId;
  final int id;
  final String title;
  final String body;

  Post(this.userId, this.id, this.title, this.body);

  String toJson() {
    // Necessário converter para Map<String, dynamic>
    Map<String, dynamic> input = { "userId": this.userId, "id": this.id,
                                   "title": this.title, "body": this.body, };

    // Converte para JSON
    return jsonEncode(input);
  }

  static Post fromJson(String jsonSource) {
    // Converte para Map<String, dynamic>
    Map<String, dynamic> item = jsonDecode(jsonSource);
    // Cria um Post de um JSON
    return Post(item['userId'], item['id'], item['title'], item['body']);
  }
}
```



Requisição HTTP com Flutter

Alguns cuidados!

Não faça isso!!

O que é absolutamente ERRADO é buscar dados HTTP, ou acionar qualquer outro método, dentro do **Widget build()**:

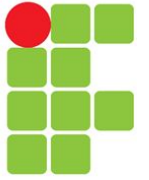
- Build é invocado frequentemente para reconstruir a tela. Portanto, teríamos várias requisições http se colocássemos `execute()` no build.

Armazenando o Future em cache

- Ao invés, utilize o método **initState** de **StatefulWidget**. Ele é executado apenas uma vez, assim a chamada da API também será executada apenas uma vez.

Exemplo

```
class MyState extends State<MyWidget> {  
    var data;  
  
    @override  
    void initState() {  
        super.initState();  
        fetchData(); // Chama a função que faz o GET  
    }  
    // Função separada para lidar com o async/await  
    Future<void> fetchData() async {  
        final response = await http.get(  
            Uri.parse('https://jsonplaceholder.typicode.com/posts/1'),  
        );  
  
        if (response.statusCode == 200) {  
            setState(() {  
                // Atualiza o estado com os dados recebidos  
                data = Post.fromJson(response.body);  
            });  
        }  
    }  
    @override  
    Widget build(BuildContext context) {  
        if (data == null) {  
            return const Center(child: CircularProgressIndicator());  
        }  
        return Text((data as Post).toJson());  
    }  
}
```



Exercícios

Para as atividades a seguir, inicie seguindo o tutorial do link:

<https://dart.dev/libraries/async/async-await>

Atividade 1: Realize a prática usando **async** e **await** do link a seguir (traduzir)

<https://dart.dev/codelabs/async-await#exercise-practice-using-async-and-await>

Atividade 2: Realize a prática usando tratamento de erros em operações assíncronas do link a seguir (traduzir)

<https://dart.dev/codelabs/async-await#exercise-practice-handling-errors>



Exercícios

Atividade 3: Continue o projeto Flutter sobre requisições HTTP e implemente os métodos GET (para recuperar uma lista), PUT e DELETE aos serviços do **jsonplaceholder**.

Em seguida, crie uma interface gráfica simples (com o layout que achar adequado) para realizar todas as operações com o serviço **jsonplaceholder**: GET, GET (lista), POST, PUT e DELETE.



Referências

MIOLA, Alberto. **Flutter Complete Reference: Create Beautiful, Fast and Native Apps for Any Device.** In:____. Network. Independently published, 2020.

<https://dart.dev/codelabs/async-await>

<https://dart.dev/tutorials/language/streams>

<https://dart.dev/articles/libraries/creating-streams>

<https://dart.dev/tutorials/server/fetch-data>